

Midterm Review

Zijun Yao

Assistant Professor, EECS Department
The University of Kansas

Scope

- Time: March 10th 4:00pm – 5:15pm
- Location: LEEP2 2300 (Classroom for lectures)
- Important Instruction:
 - 4 questions in total (linear regression, neural networks, CNN, RNN).
GNN is not included
 - Homework is the best preparation source, solution will be on canvas.
 - Open-book - lecture slides, notes, books are allowed. Calculator is allowed. No any other electronic devices.
 - **Write clearly, make explanation (can earn your points in case your answer is wrong).**
 - **Don't get stuck at particular problems. Finish all and come back later.**
 - **Please be well prepared, zero tolerance for cheating, seats will be randomized.**

Midterm Exam

1. Linear regression: model definition, loss function, gradient descent, regularization
 2. Neural network: fully-connected layers, forward propagation, loss, backward propagation
 3. CNN: convolutional layer, pooling layer
 4. RNN: basic RNN, memory update, pro/con
- Computational questions for 1 and 2. Conceptual questions for 3 and 4.

Agenda

- Linear Regression
- Neural Networks
- CNN
- RNN

Step 1: Model definition

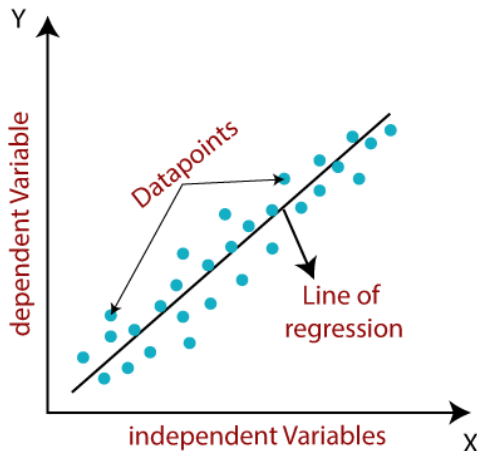
Parameters

Bias: a fixed offset

Weights: significance of each feature

$$\hat{y} = b + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

Predicted value of y



Price of House
\$127,325
\$156,570
\$158,895
\$200,025
\$230,384
\$233,835
\$261,420
\$433,500

Target

Size of House
950
1,535
1,605
1,905
2,057
2,227
3,150
3,620

1st Feature

Lot Size (acre)
2.5
1.5
2.25
2.5
2.25
2.75
1
3

2nd Feature

of Bathrooms
1
2
1.5
1.5
2
2
2
3

d -th Feature

Data

A linear relationship between feature and target

Step 1: A variant form of linear model

$$\hat{y} = w_0x_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$$



Dummy
1
1
1
1
1
1
1
1
1

0st Feature

Size of House
950
1,535
1,605
1,905
2,057
2,227
3,150
3,620

1st Feature

Lot Size (acre)
2.5
1.5
2.25
2.5
2.25
2.75
1
3

2nd Feature

of Bathrooms
1
2
1.5
1.5
2
2
2
3

d -th Feature

Linear Regression model

Prediction

Parameters

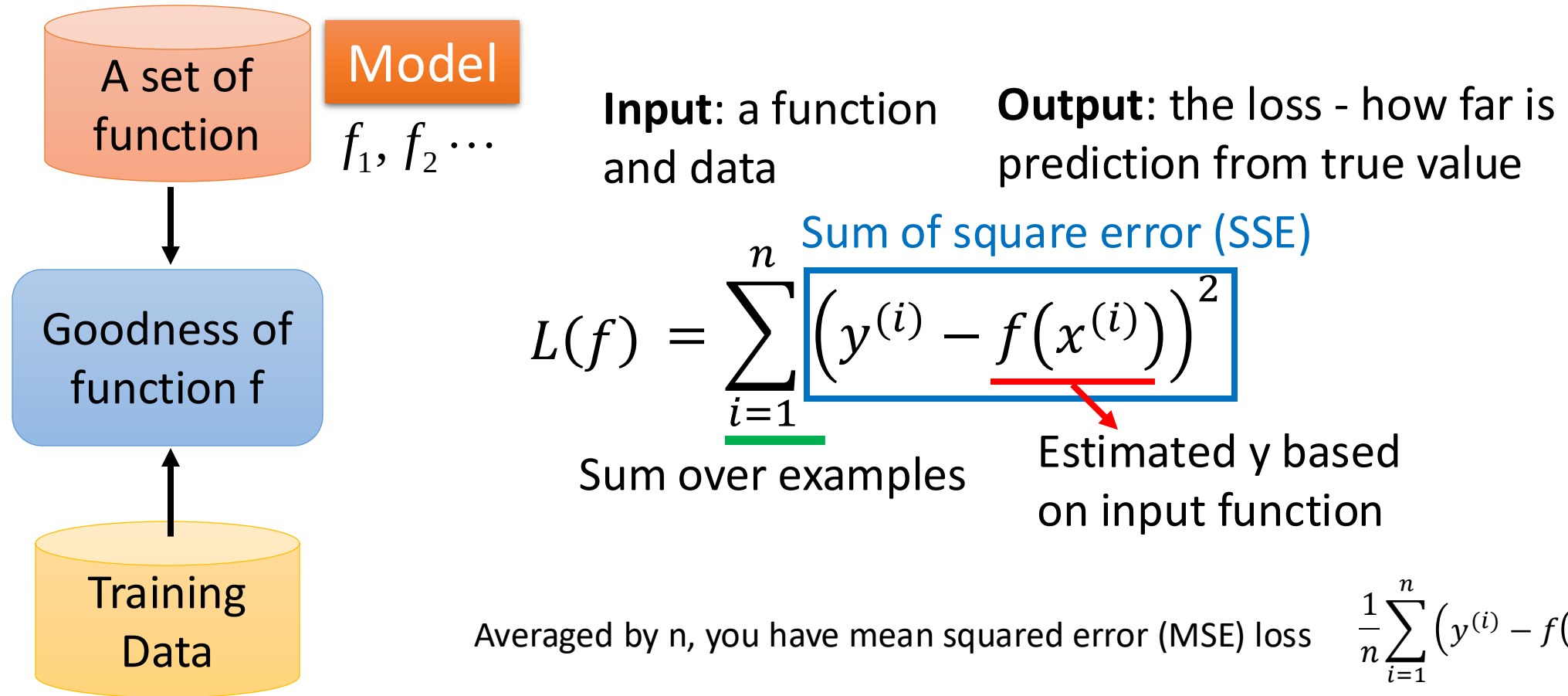
Function of $\mathcal{X}$

Features

$$f(x) = \hat{y} = \sum_{i=0}^d w_i x_i$$

Step 2: Measure error

How good is a function? - **Loss function** L



Step 2: Measure error

How good is a function? - **Loss function** L

Input: a function and data

output: the loss - how far is prediction from true value

$$L(f) = \sum_{i=1}^n \left(\underline{y^{(i)}} - \underline{f(x^{(i)})} \right)^2$$

Sum over examples

True y

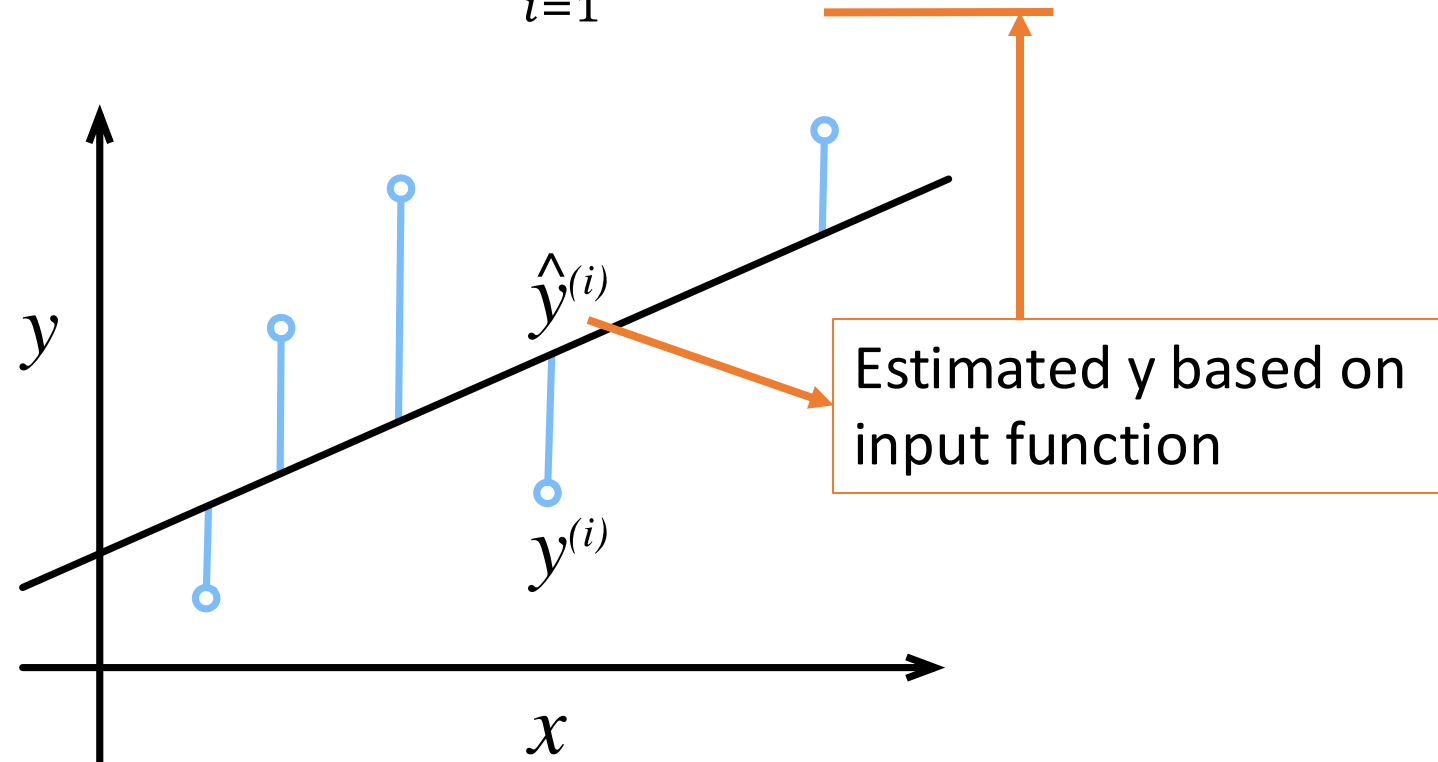
Estimated y based
on input function



$$\begin{aligned} &L(w_{size}, w_{age}, b) \\ &= \sum_{i=1}^n \left(\underline{y^{(i)}} - \underline{\left(b + w_{size} \cdot x_{size}^{(i)} + w_{age} \cdot x_{age}^{(i)} \right)} \right)^2 \end{aligned}$$

Step 2: Loss function

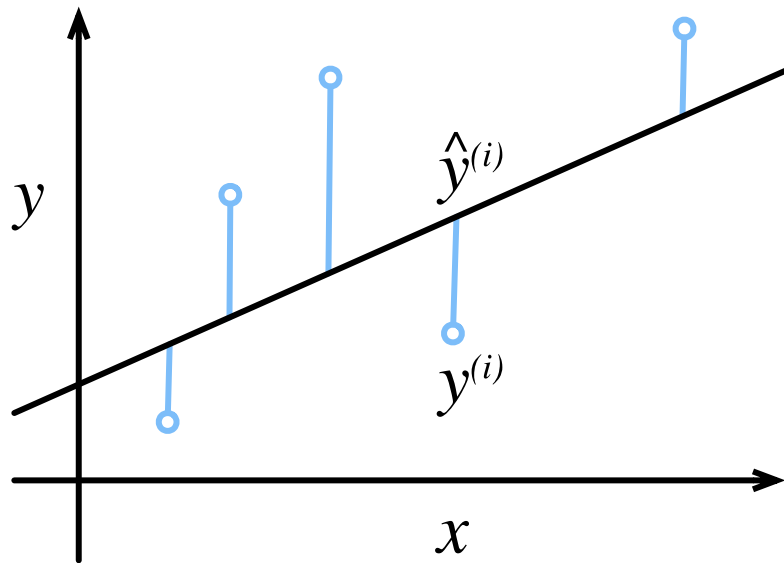
$$L(f) = \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}) \right)^2$$



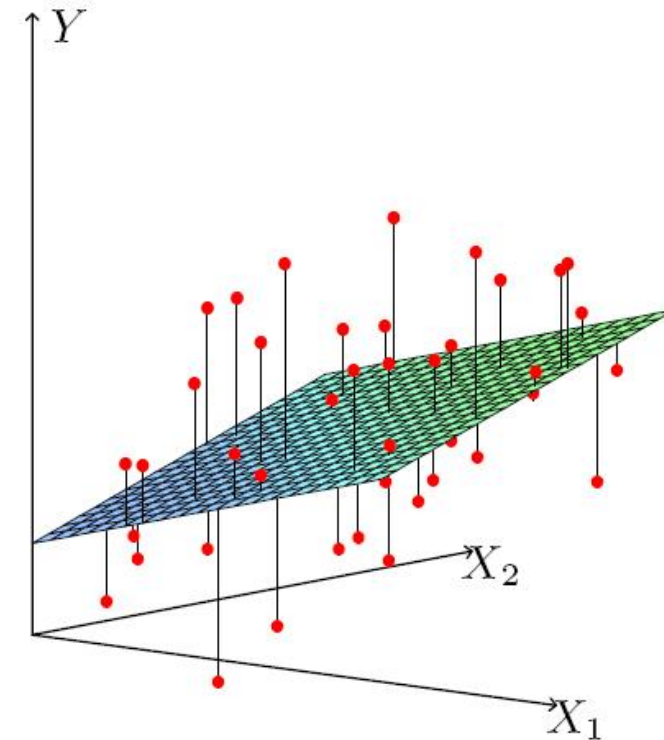
A simple case where only one feature is used to predict y

Step 2: Loss function

$$L(f) = \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}) \right)^2$$

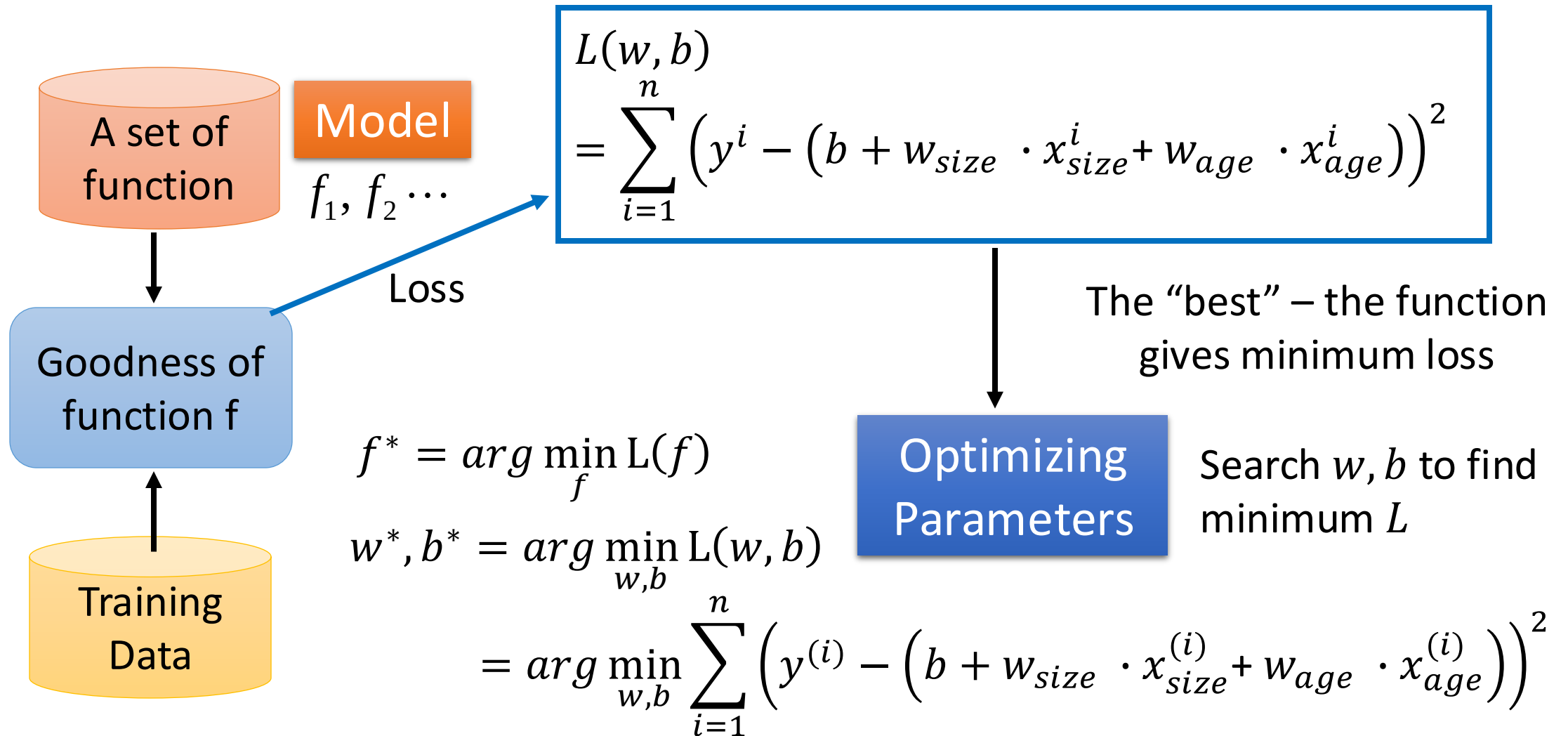


A simple case where only one feature is used to predict y



When there are 2 features to predict y

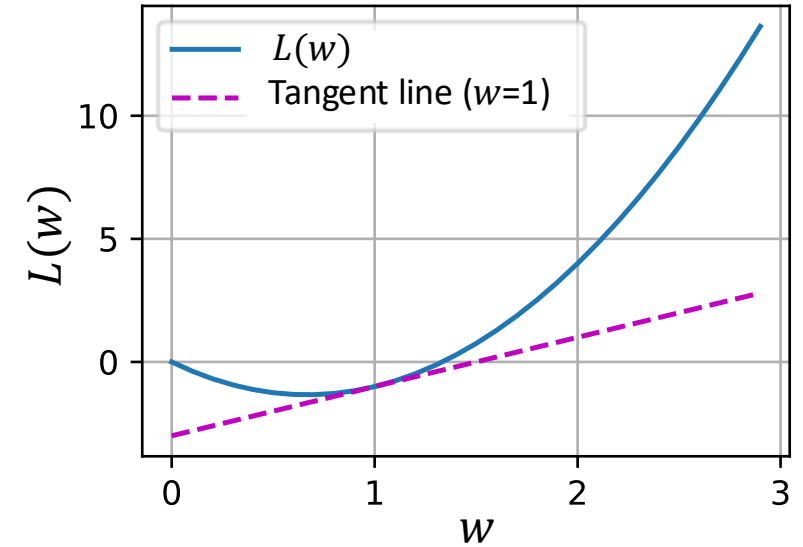
Step 3: Find the best function



Derivatives

- The *derivative* of loss function L is

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$



The sensitivity to change of loss function with respect to a change in a parameter w

Derivatives

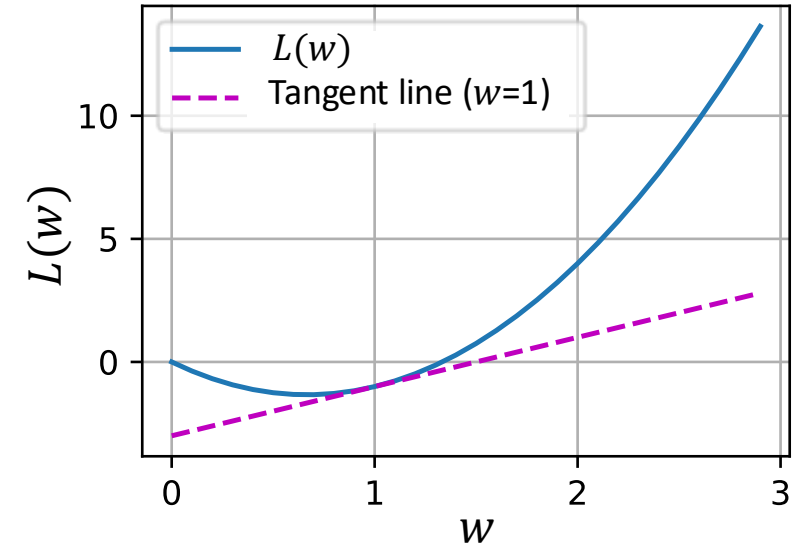
- The *derivative* of loss function L is

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$

- Partial Derivatives: Let $L(w)$ be loss function of parameters

$$\frac{\partial L(w)}{\partial w_i} = \lim_{h \rightarrow 0} \frac{L(w_0, \dots, w_{i-1}, w_i + h, w_{i+1}, \dots, w_d) - L(w_0, \dots, w_{i-1}, w_i, w_{i+1}, \dots, w_d)}{h}$$

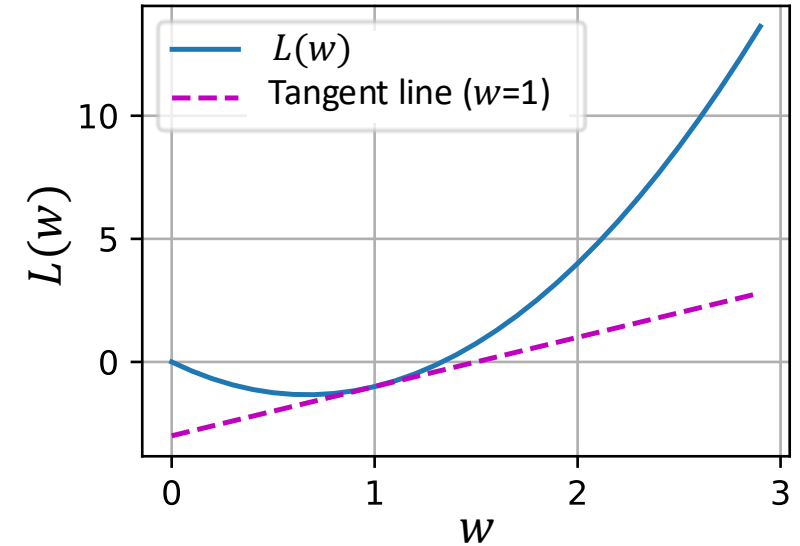
Function derivative with respect to one of parameters w_i , with the others held constant



Derivatives

- The *derivative* of loss function L is

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$



- Partial Derivatives: Let $L(\mathbf{w})$ be loss function of parameters

$$\frac{\partial L(\mathbf{w})}{\partial w_i} = \lim_{h \rightarrow 0} \frac{L(w_0, \dots, w_{i-1}, x_i + h, x_{i+1}, \dots, w_d) - L(w_0, \dots, w_{i-1}, x_i, x_{i+1}, \dots, w_d)}{h}$$

- Gradients

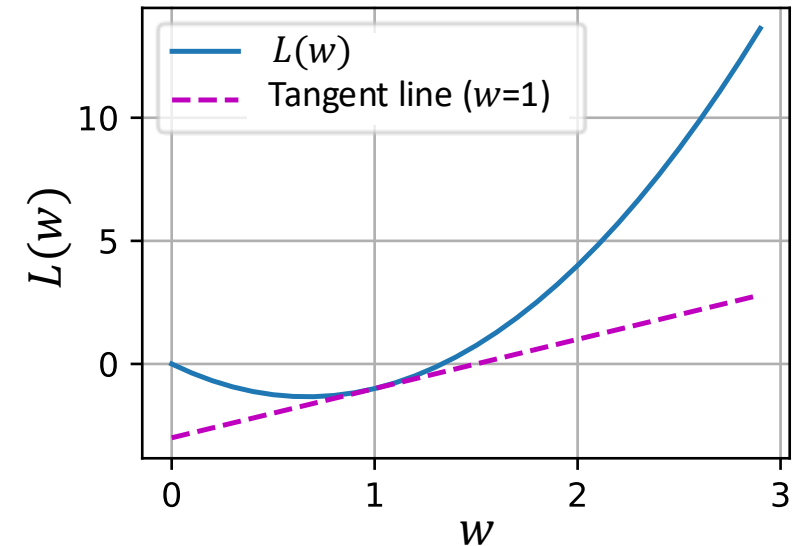
$$\nabla L(\mathbf{w}) = \left[\frac{\partial L(\mathbf{w})}{\partial w_0}, \frac{\partial L(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial L(\mathbf{w})}{\partial w_d} \right]^T$$

Gradients is a vector consists of partial derivative of each parameter

Derivatives

- The *derivative* of loss function L is

$$L'(w) = \lim_{h \rightarrow 0} \frac{L(w + h) - L(w)}{h}$$



- Partial Derivatives: Let $L(\mathbf{w})$ be loss function of parameters

$$\frac{\partial L(\mathbf{w})}{\partial w_i} = \lim_{h \rightarrow 0} \frac{L(w_0, \dots, w_{i-1}, w_i + h, w_{i+1}, \dots, w_d) - L(w_0, \dots, w_{i-1}, w_i, w_{i+1}, \dots, w_d)}{h}$$

- Gradients

$$\nabla L(\mathbf{w}) = \left[\frac{\partial L(\mathbf{w})}{\partial w_0}, \frac{\partial L(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial L(\mathbf{w})}{\partial w_d} \right]^T$$

Gradient is a direction that increase the value of $L(\mathbf{w})$

How to reduce Loss function?

Subtract gradient from each parameter w

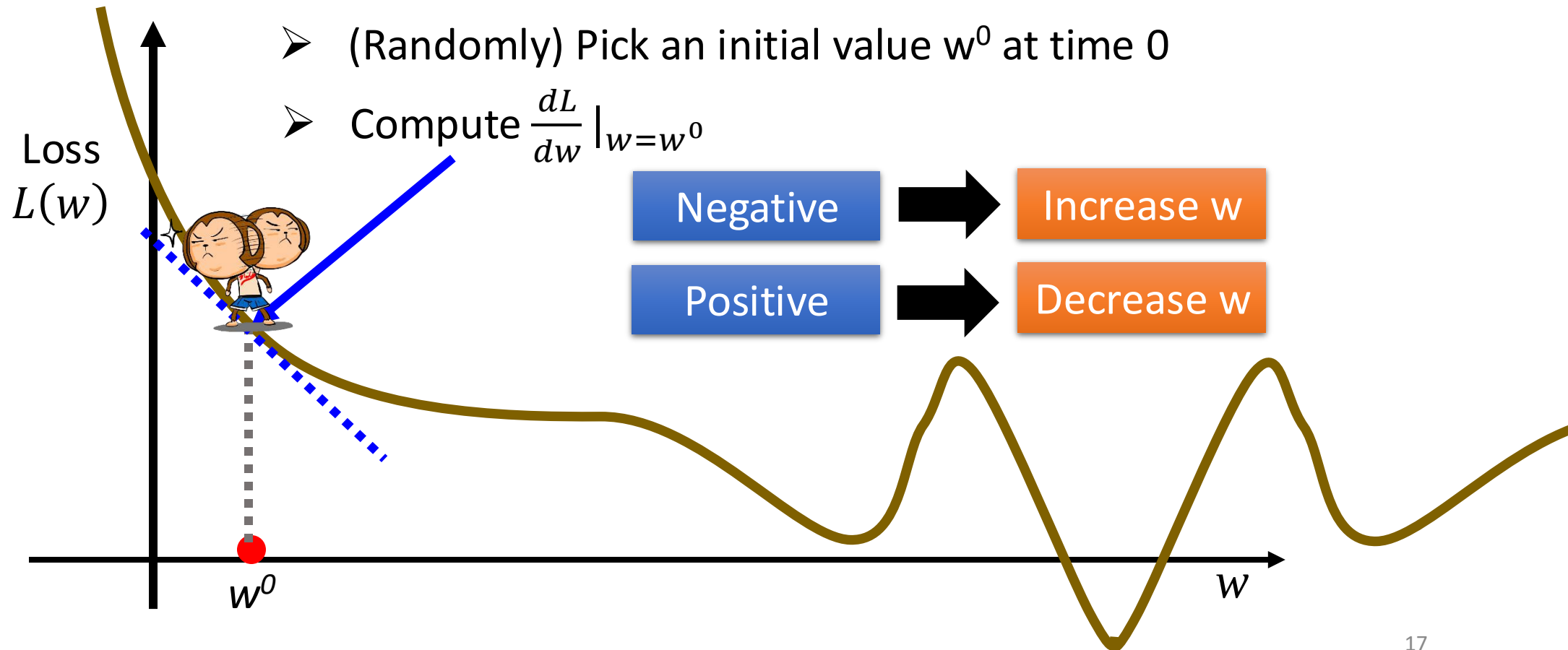
Step 3: Gradient descent

- Gradient of Linear Regression

$$\begin{aligned}\frac{\partial L}{\partial w} &= \frac{\partial}{\partial w} \sum_{i=1}^n (y^{(i)} - f(x^{(i)}))^2 \\ &= \frac{\partial}{\partial w} \sum_{i=1}^n (y^{(i)} - \sum_{j=0}^d w_j x_j^{(i)})^2 \\ &= \sum_{i=1}^n 2(y^{(i)} - \sum_{j=0}^d w_j x_j^{(i)}) \times \frac{\partial}{\partial w} (y^{(i)} - \sum_{j=0}^d w_j x_j^{(i)}) \\ &= - \sum_{i=1}^n 2(y^{(i)} - \sum_{j=0}^d w_j x_j^{(i)}) x_j^{(i)}\end{aligned}$$

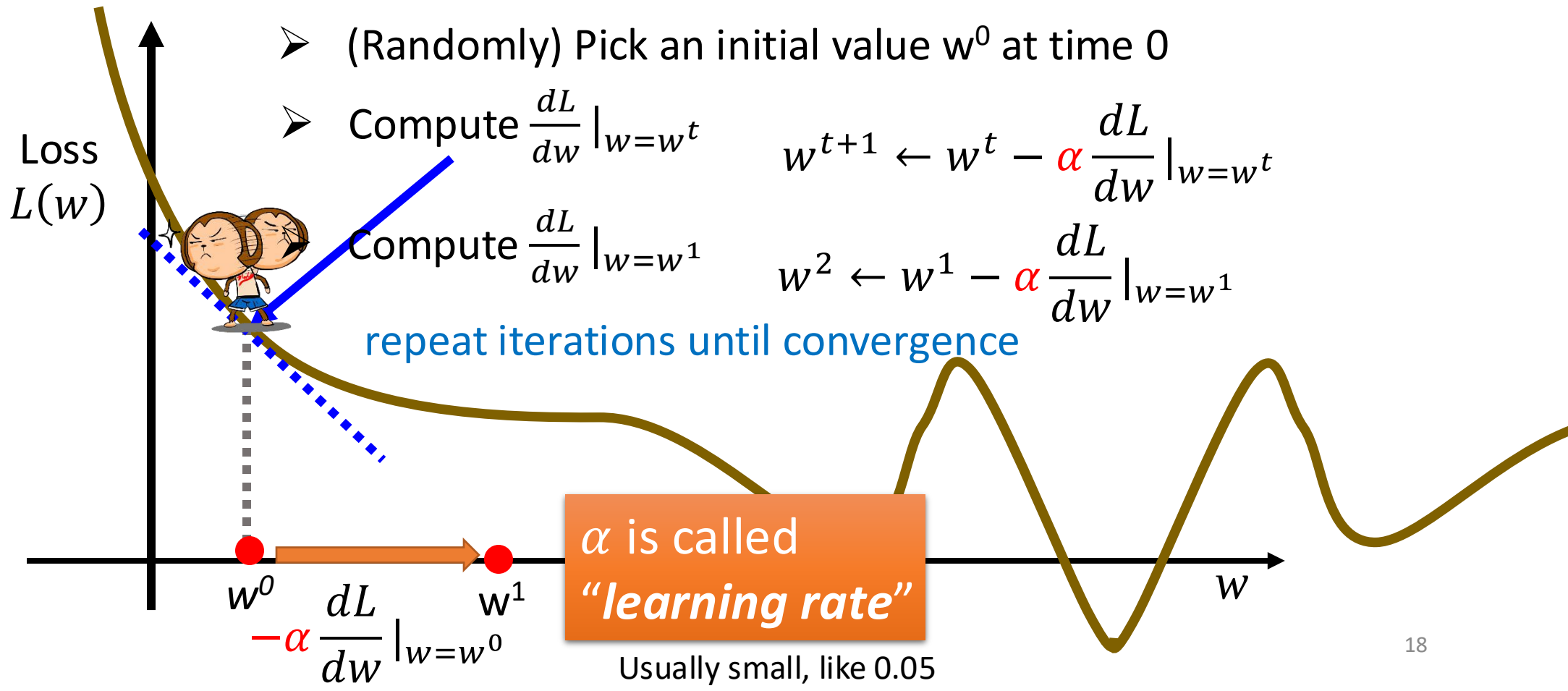
Step 3: Gradient descent

- Consider loss function $L(w)$ with one parameter w : $w^* = \arg \min_w L(w)$



Step 3: Gradient descent

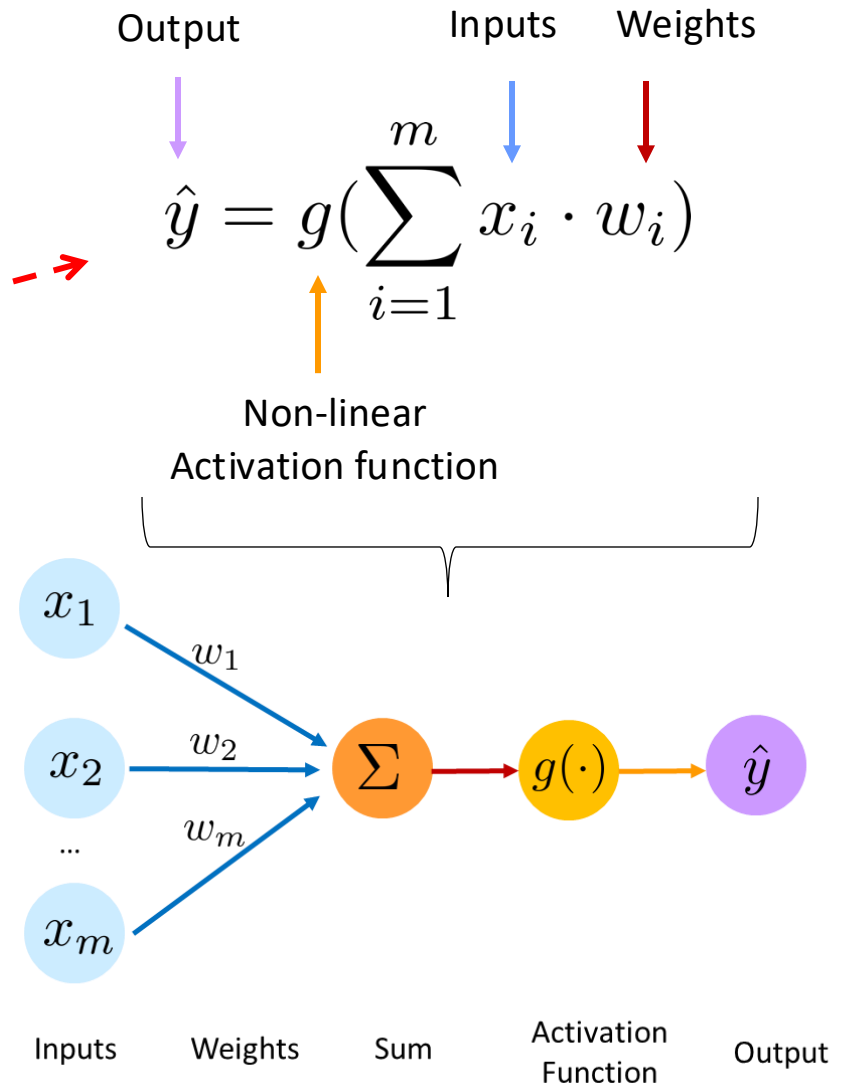
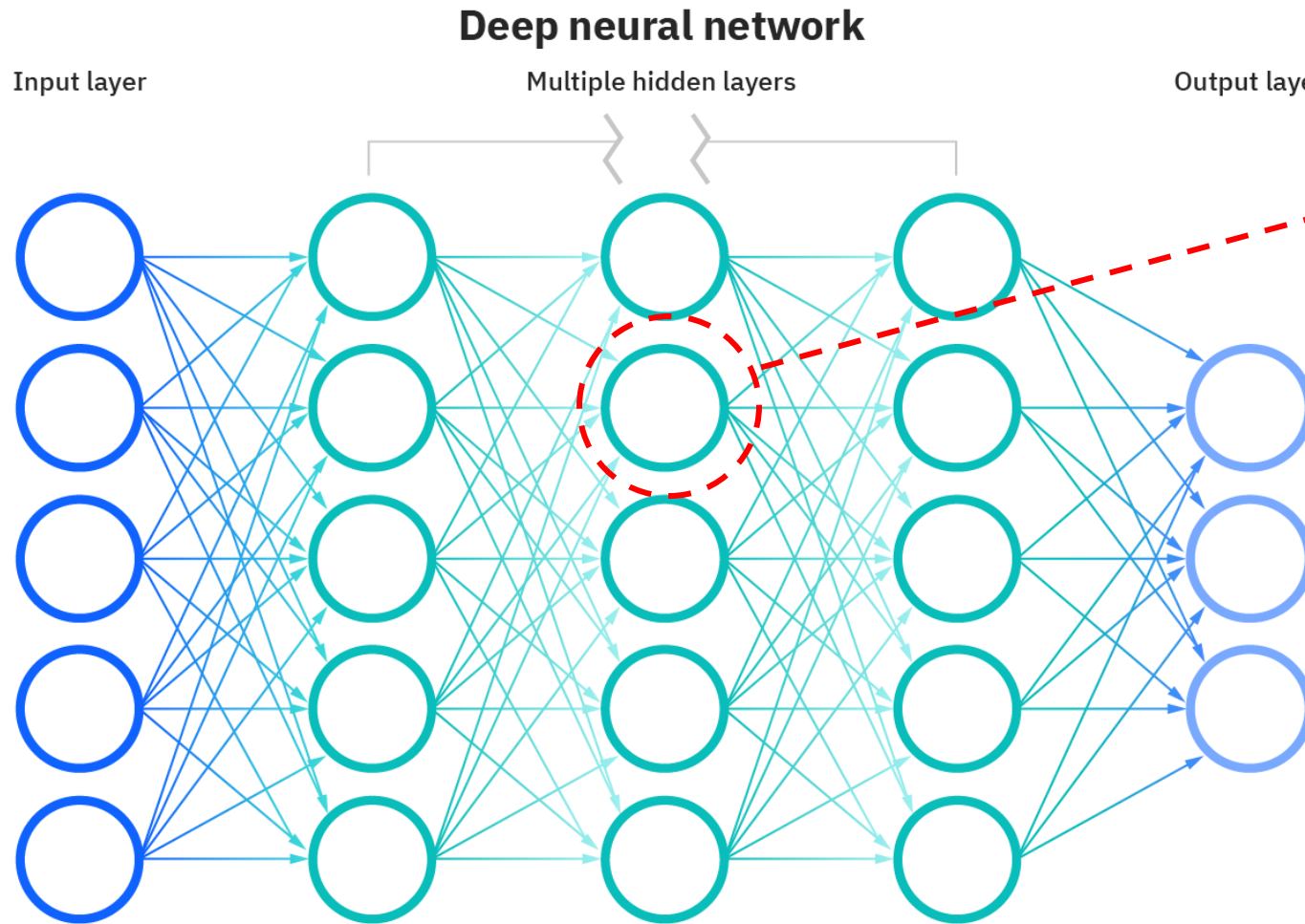
- Consider loss function $L(w)$ with one parameter w : $w^* = \arg \min_w L(w)$



Agenda

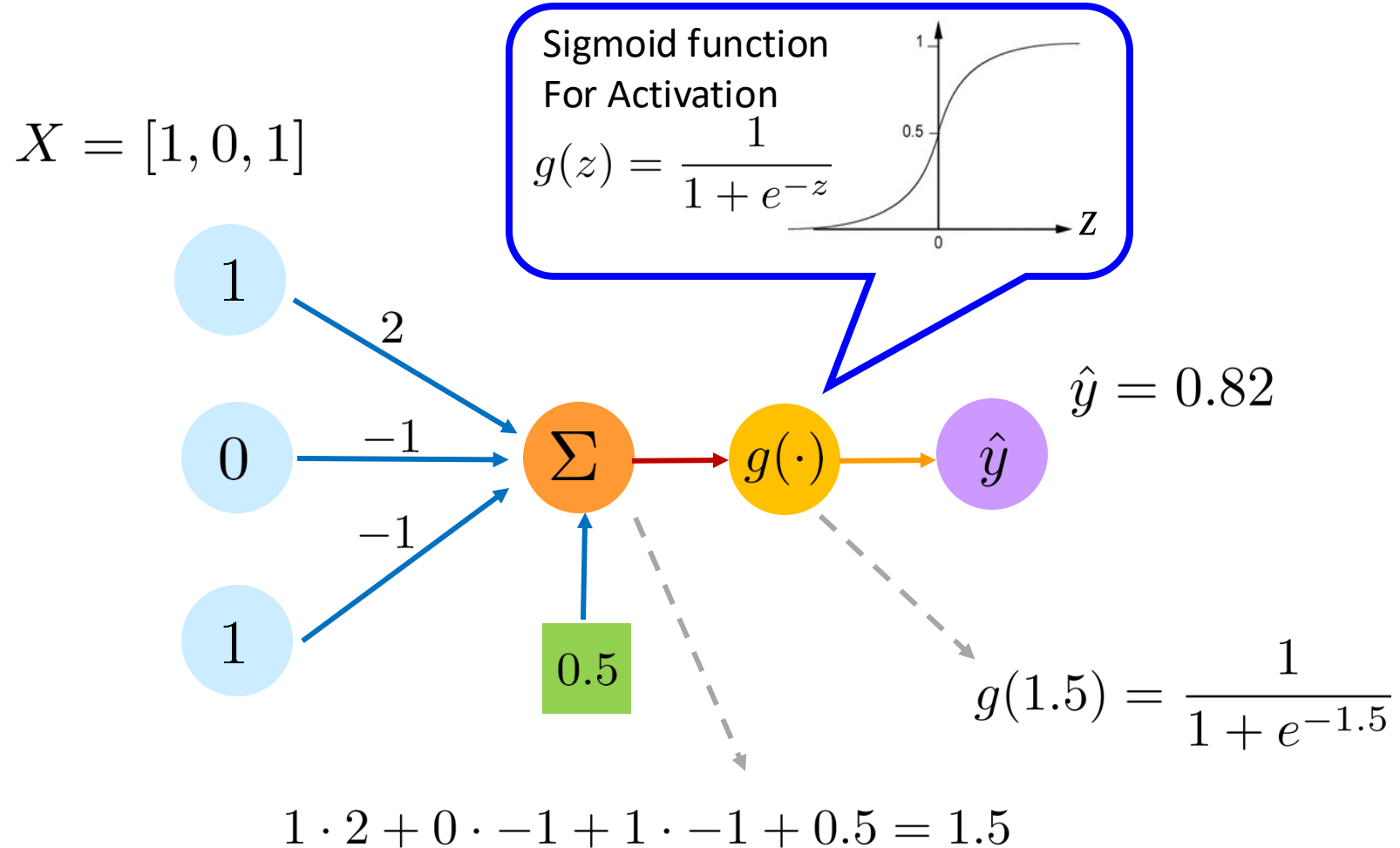
- Linear Regression
- **Neural Networks**
- CNN
- RNN

Zoom-in a single neuron



Detail steps within a single neuron

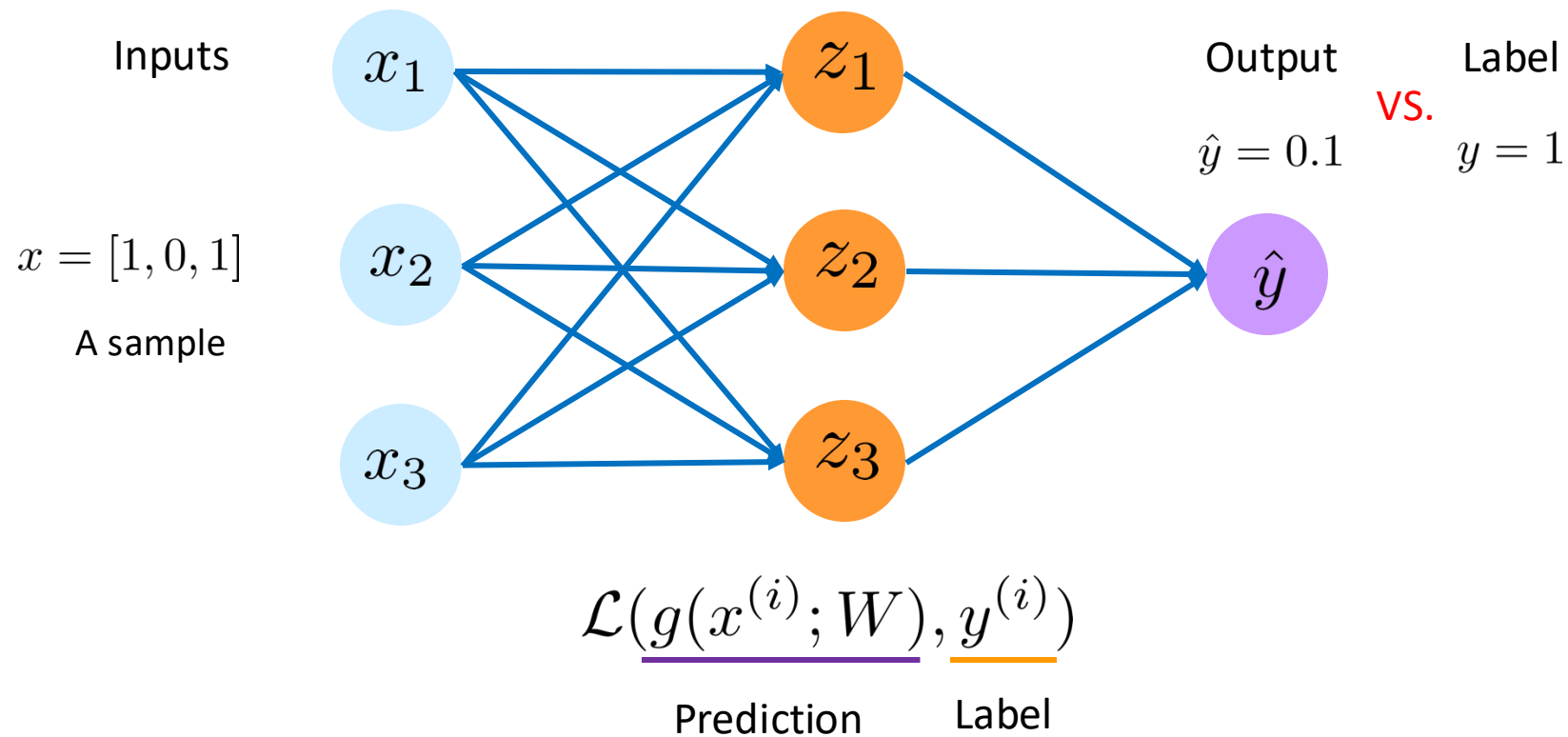
Example of a single neuron



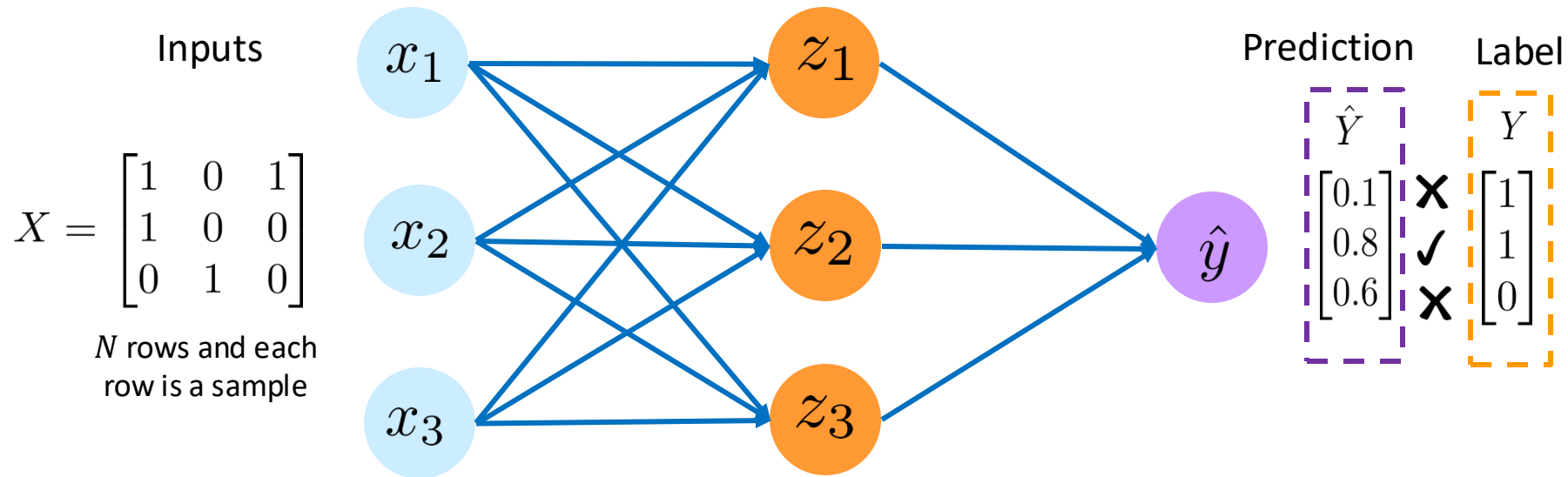
Measuring loss

Loss of the network

- Measures the cost incurred at output (prediction)



Cross entropy loss



- Measures the loss for binary classification where output is a probability between 0 and 1

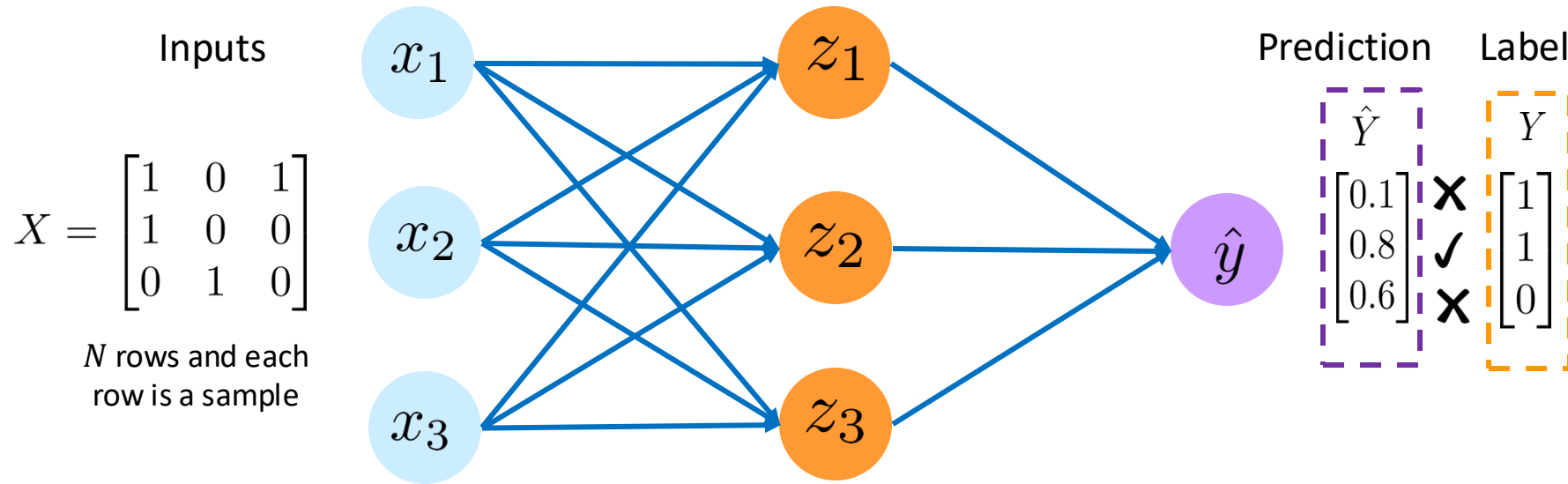
$$J(W) = -\frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Label}} \underbrace{\log(g(x^{(i)}; W))}_{\text{Prediction}} + \underbrace{(1 - y^{(i)})}_{\text{Label}} \underbrace{\log(1 - g(x^{(i)}; W))}_{\text{Prediction}}$$

Sample with positive label Sample with negative label

PyTorch

```
loss = torch.nn.BCEWithLogitsLoss()
output = loss(predicted, y, reduction='mean')
```

Mean Squared Error (MSE) loss



- Measures the loss for regression where output is a continuous real number

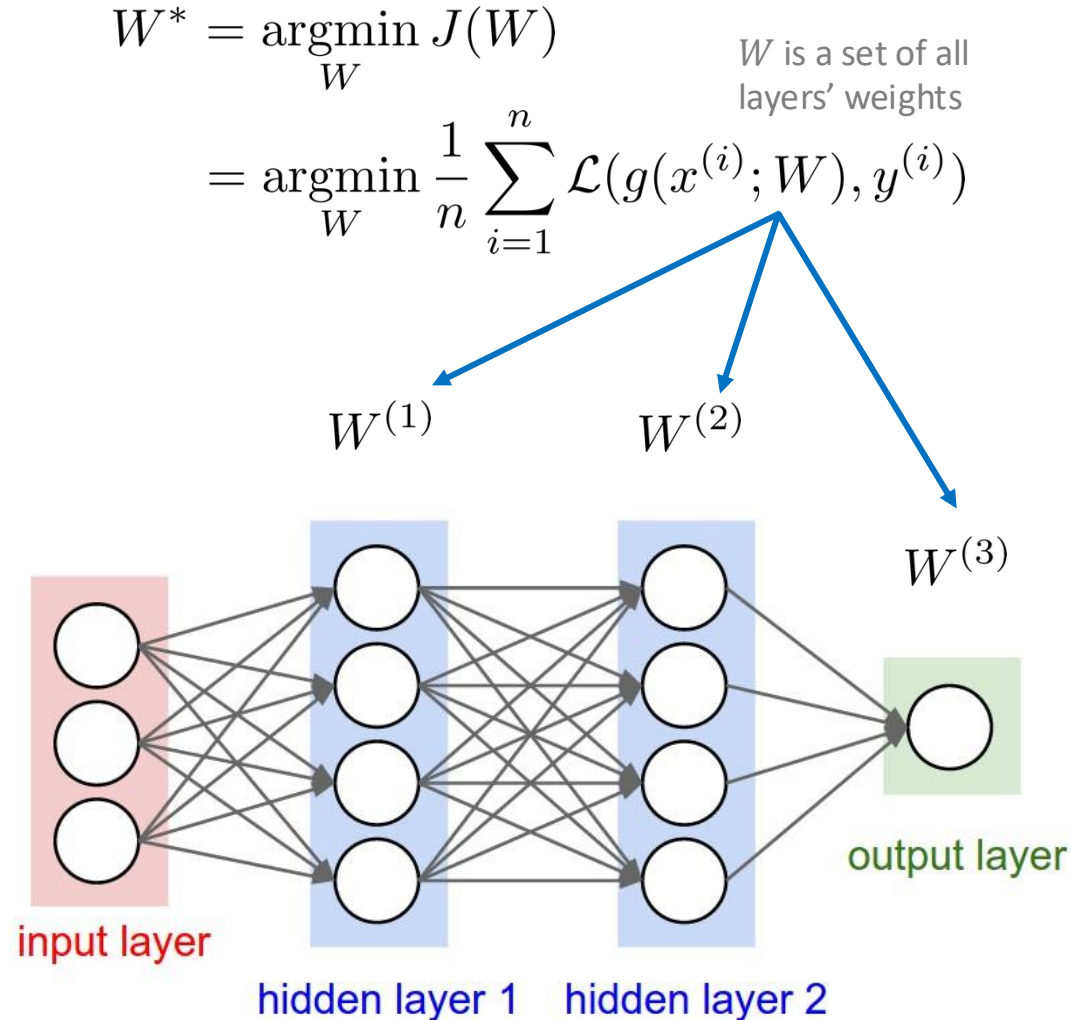
$$J(W) = \frac{1}{n} \sum_{i=1}^n (\underbrace{y^{(i)}}_{\text{Label}} - \underbrace{g(x^{(i)}; W)}_{\text{Prediction}})^2$$

PyTorch

```
loss = torch.nn.MSELoss()  
output = loss(predicted, y, reduction='mean')
```

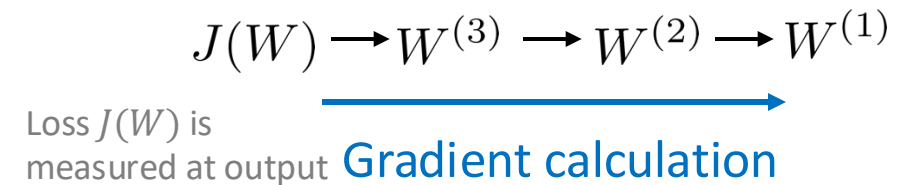
Update parameters to optimize loss

- **Goal:** find the network weights that achieve the lowest loss



Weights in every layer need to be optimized

- Loss is calculation at output
- We need to update parameters to lower loss
- How do we decide gradient for each parameter?
- **Backpropagation!**

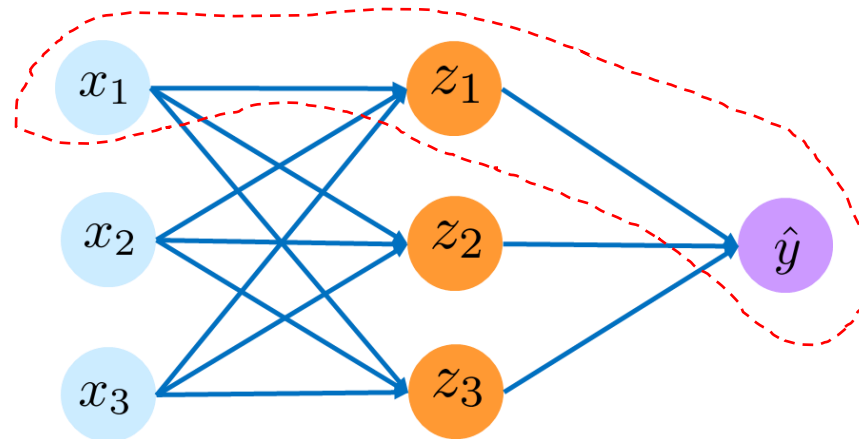
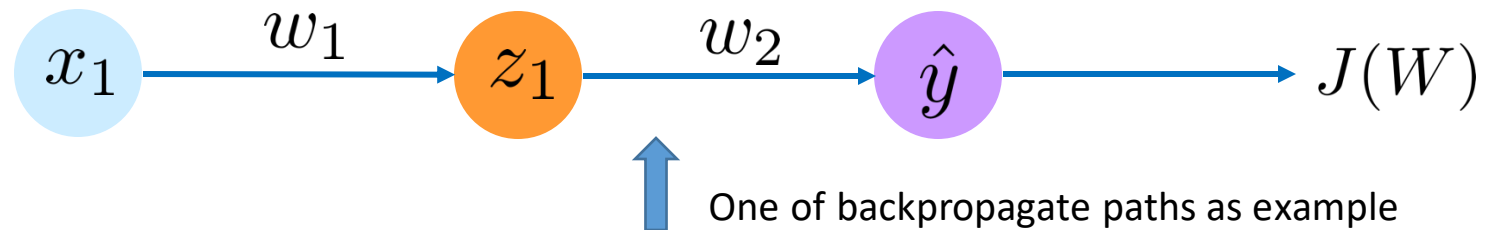


Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$\begin{array}{l} ? \\ \Delta w_1 \rightarrow \Delta J(W) \\ \Delta w_2 \rightarrow \Delta J(W) \end{array}$$

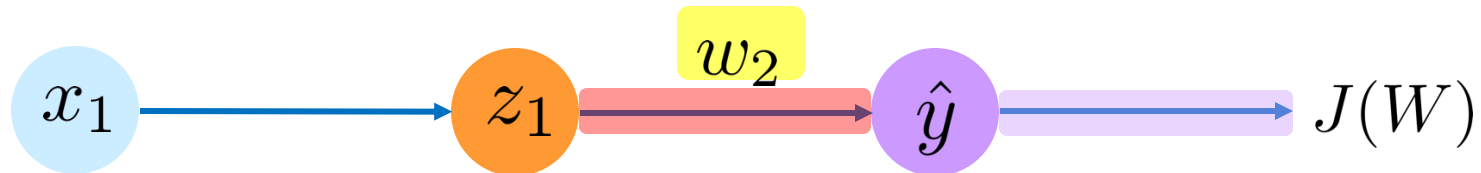


Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$? \quad \Delta w_2 \rightarrow \Delta J(W)$$



$$\frac{\partial J(W)}{\partial w_2} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} \times \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_{\text{red}}$$

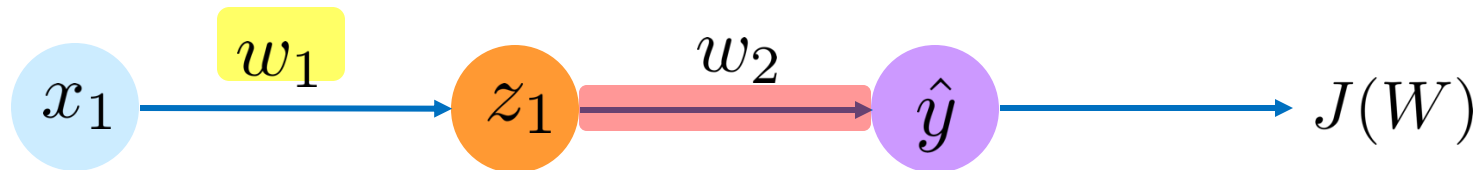
Chain rule: from back to front

Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$? \quad \Delta w_1 \rightarrow \Delta J(W)$$



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_1}$$

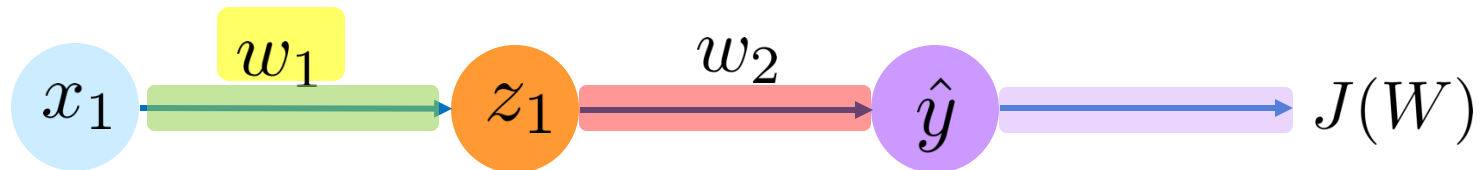
Apply chain rule

Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$? \quad \Delta w_1 \rightarrow \Delta J(W)$$



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} \times \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} \times \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{green}}$$

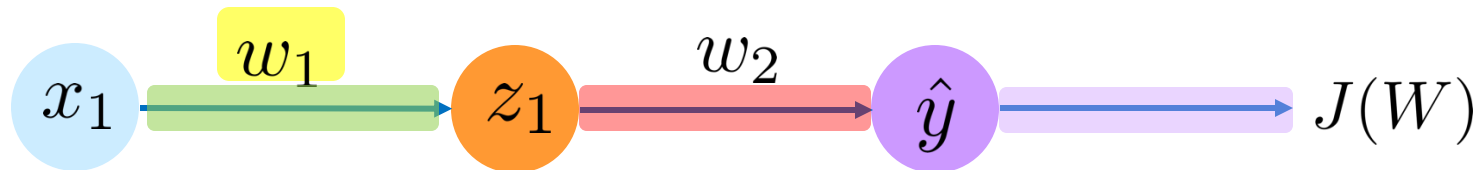
Chain rule: one more layer backward

Backpropagation

Compute gradient

- How does a small change in one weight affect the final loss?

$$? \quad \Delta w_1 \rightarrow \Delta J(W)$$

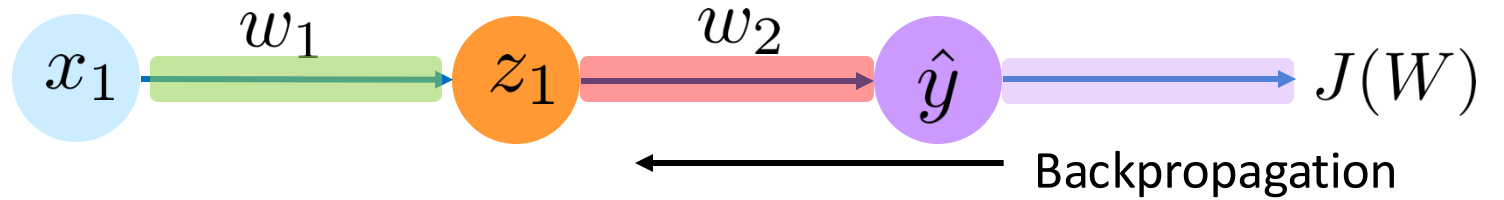


$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} \times \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} \times \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{green}}$$

Repeat this “message passing” for **every weight** in each layer using the gradients from **afterward layers**, until all the layers have been updated by loss.

Gradients by partial derivative

- Cross Entropy Loss



$$J(W) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log(\hat{y}) + (1 - y^{(i)}) \log(1 - \hat{y}) \quad \text{Each neuron } f(x) = \frac{1}{1 + e^{-w^\top x}}$$

Partial derivative of **loss** respect to $\hat{y}$

$$\begin{aligned} \frac{\partial J(W)}{\partial \hat{y}} &= -(y \times \frac{1}{\hat{y}} + (1 - y) \times \frac{1}{1 - \hat{y}} \times -1) \\ &= \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \end{aligned}$$

Gradient for w_2

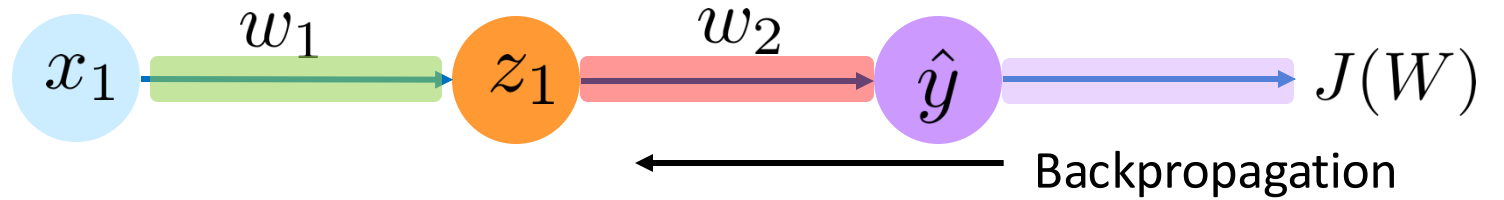
$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_2}$$

Partial derivative of $\hat{y}$ respect to a weight w_2

$$\begin{aligned} \frac{\partial \hat{y}}{\partial w_2} &= -1 \times (1 + e^{-w_2^\top z_1})^{-2} \times e^{-w_2^\top z_1} \times -z_1 \\ &= \frac{1}{1 + e^{-w_2^\top z_1}} \times \frac{e^{-w_2^\top z_1}}{1 + e^{-w_2^\top z_1}} \times z_1 \\ &= \hat{y}(1 - \hat{y})z_1 \end{aligned}$$

Gradients by partial derivative

- Cross Entropy Loss



$$J(W) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log(\hat{y}) + (1 - y^{(i)}) \log(1 - \hat{y}) \quad \text{Each neuron } f(x) = \frac{1}{1 + e^{-w^\top x}}$$

Partial derivative of **loss** respect to $\hat{y}$

$$\begin{aligned} \frac{\partial J(W)}{\partial \hat{y}} &= -(y \times \frac{1}{\hat{y}} + (1 - y) \times \frac{1}{1 - \hat{y}} \times -1) \\ &= \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \end{aligned}$$

Partial derivative of $\hat{y}$ respect to a weight w_2

$$\begin{aligned} \frac{\partial \hat{y}}{\partial w_2} &= -1 \times (1 + e^{-w_2^\top z_1})^{-2} \times e^{-w_2^\top z_1} \times -z_1 \\ &= \frac{1}{1 + e^{-w_2^\top z_1}} \times \frac{e^{-w_2^\top z_1}}{1 + e^{-w_2^\top z_1}} \times z_1 \\ &= \hat{y}(1 - \hat{y})z_1 \end{aligned}$$

Partial derivative of $\hat{y}$ respect to a weight z_1

$$\begin{aligned} \frac{\partial \hat{y}}{\partial z_1} &= -1 \times (1 + e^{-w_2^\top z_1})^{-2} \times e^{-w_2^\top z_1} \times -w_2 \\ &= \hat{y}(1 - \hat{y})w_2 \end{aligned}$$

Partial derivative of $\hat{y}$ respect to a weight z_1

$$\begin{aligned} \frac{\partial z_1}{\partial w_1} &= -1 \times (1 + e^{-w_1^\top x_1})^{-2} \times e^{-w_1^\top x_1} \times -x_1 \\ &= z_1(1 - z_1)x_1 \end{aligned}$$

Gradient for w_2

$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_2}$$

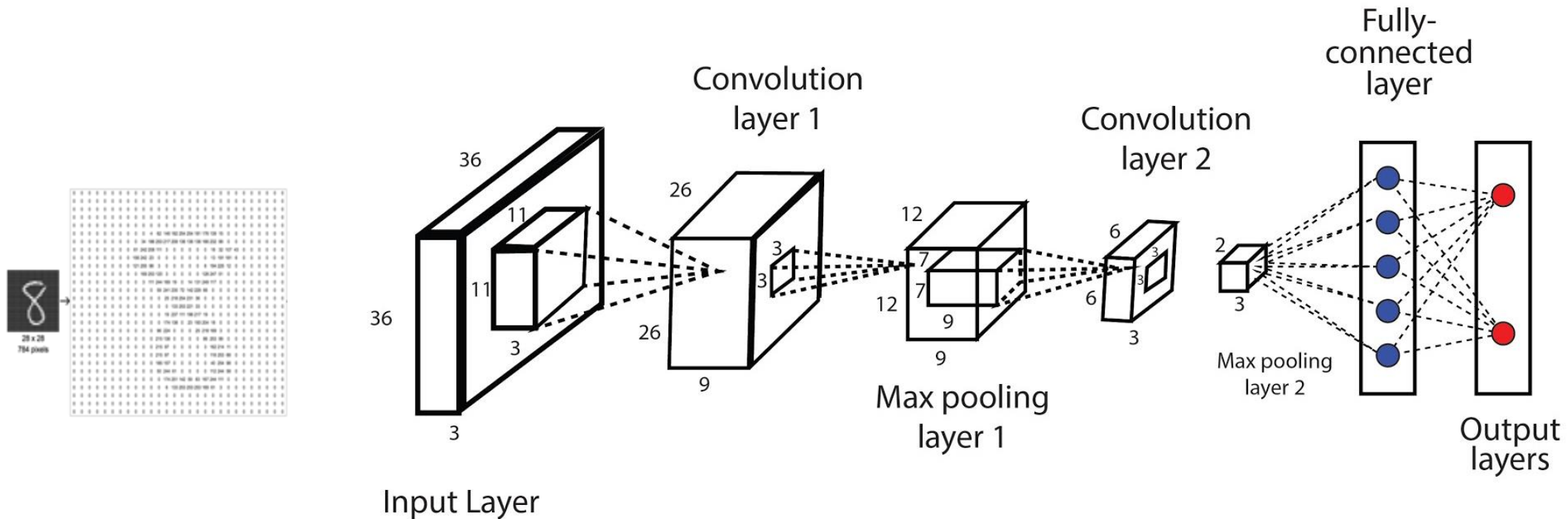
Gradient for w_1

$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$$

Agenda

- Linear Regression
- Neural Networks
- **CNN**
- RNN

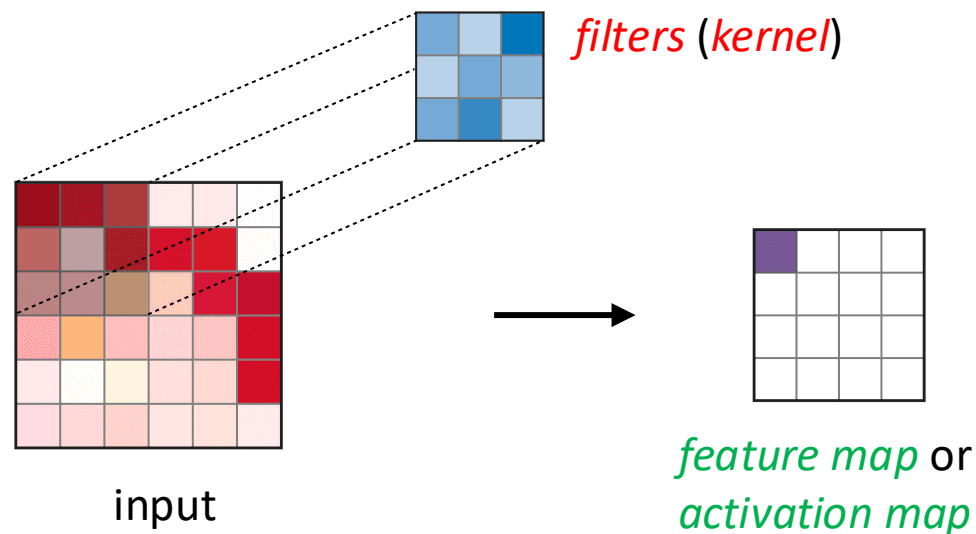
A CNN structure



Three main types of layers: **Convolutional Layer**, **Pooling Layer**, and **Fully-Connected Layer**

Convolution layer

- The convolution layer uses *filters* that perform convolution operations
 - It scan the input I with respect to its dimensions.
- Its hyperparameters include the *filter size* F and *stride* S .
- The resulting output O is called *feature map* or *activation map*.



Input		Kernel		Output																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

CNN – convolution

Those are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1
Matrix

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2
Matrix

⋮

Property 1

Each filter detects a small pattern (3 x 3).

CNN – convolution

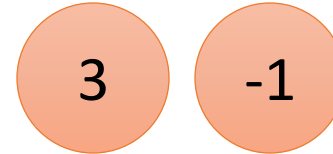
stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



CNN – convolution

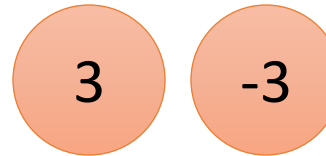
If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



We set stride=1 below

CNN – convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Property 2

CNN – Convolution

stride=1

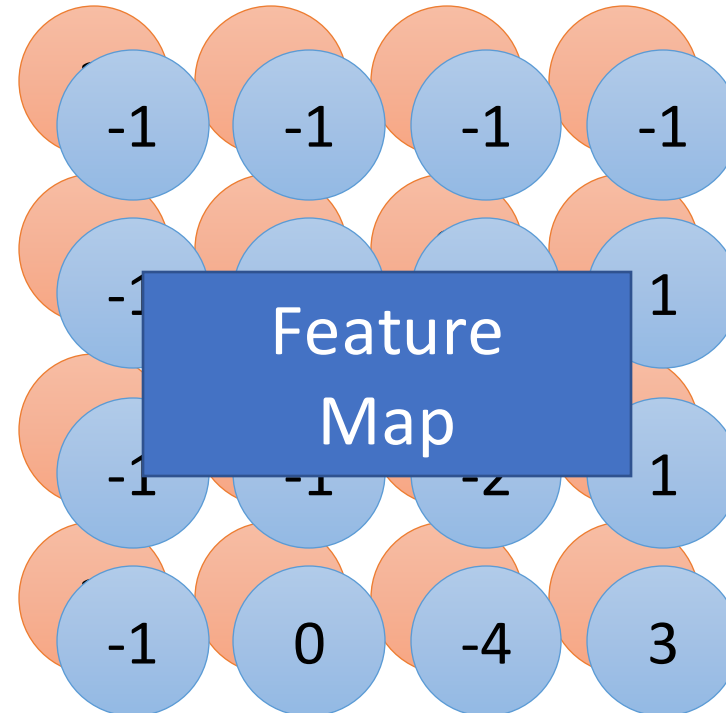
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

Do the same process for every filter



4 x 4 image

Padding and stride

- **Padding**: add extra pixels of filler around the boundary of our input image, thus increasing the effective size of the image

Input Kernel Output

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

 *

0	1
2	3

 =

0	3	8	4
9	19	25	10
21	37	43	16
6	7	8	0

- **Stride**: the number of rows and columns traversed per slide as the *stride*

Input Kernel Output

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

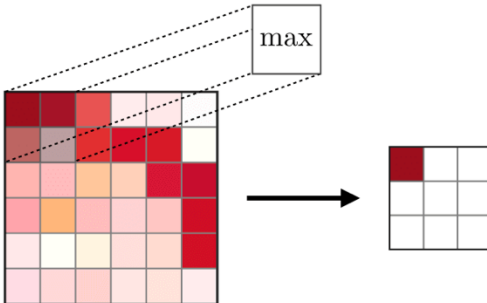
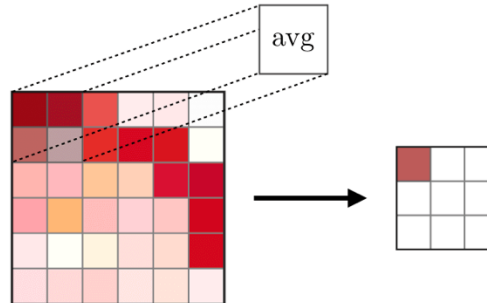
 *

0	1
2	3

 =

0	8
6	8

Pooling layer

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

The pooling layer is a downsampling operation, typically applied after a convolution layer !

CNN – max pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

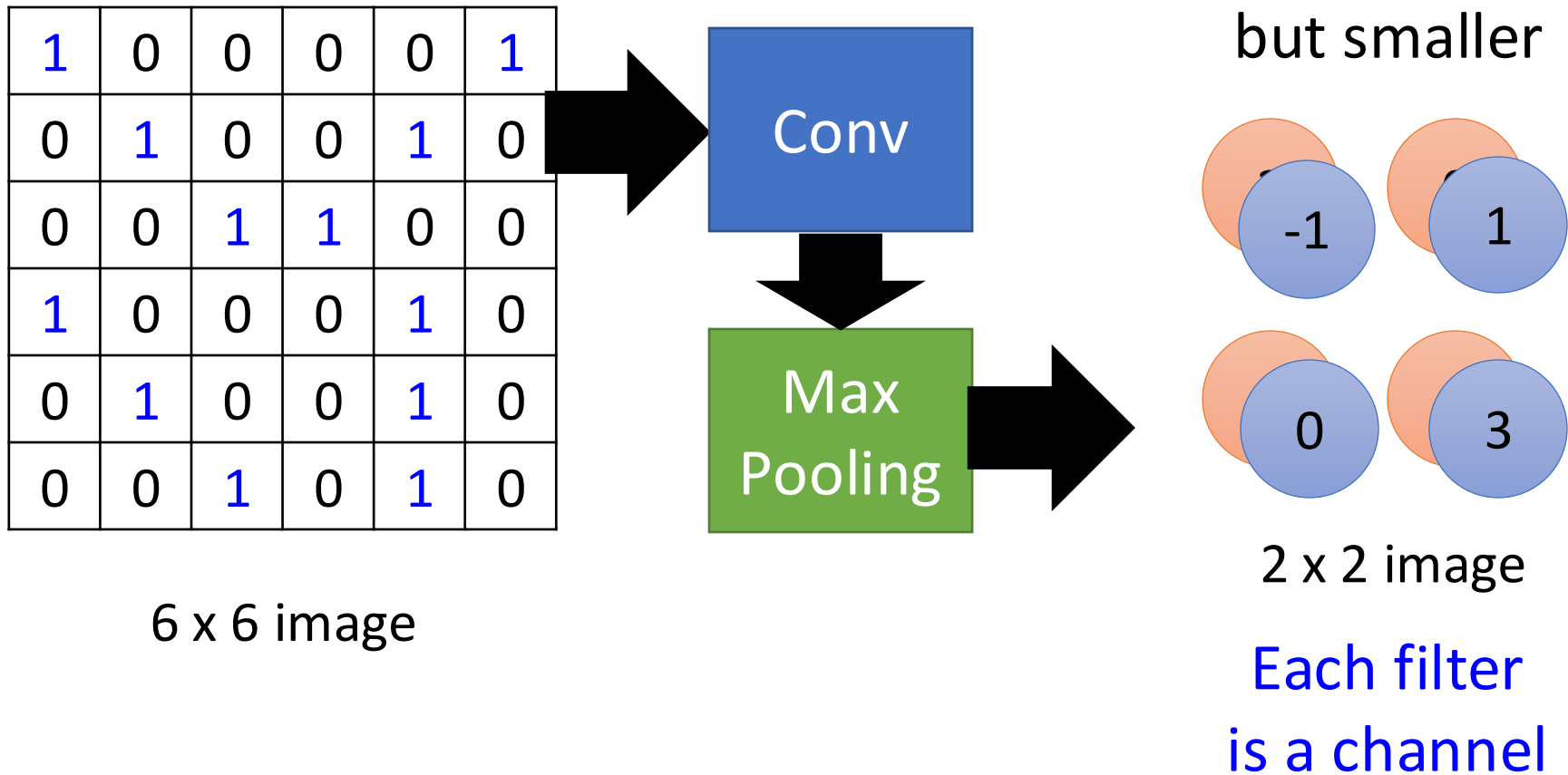
-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

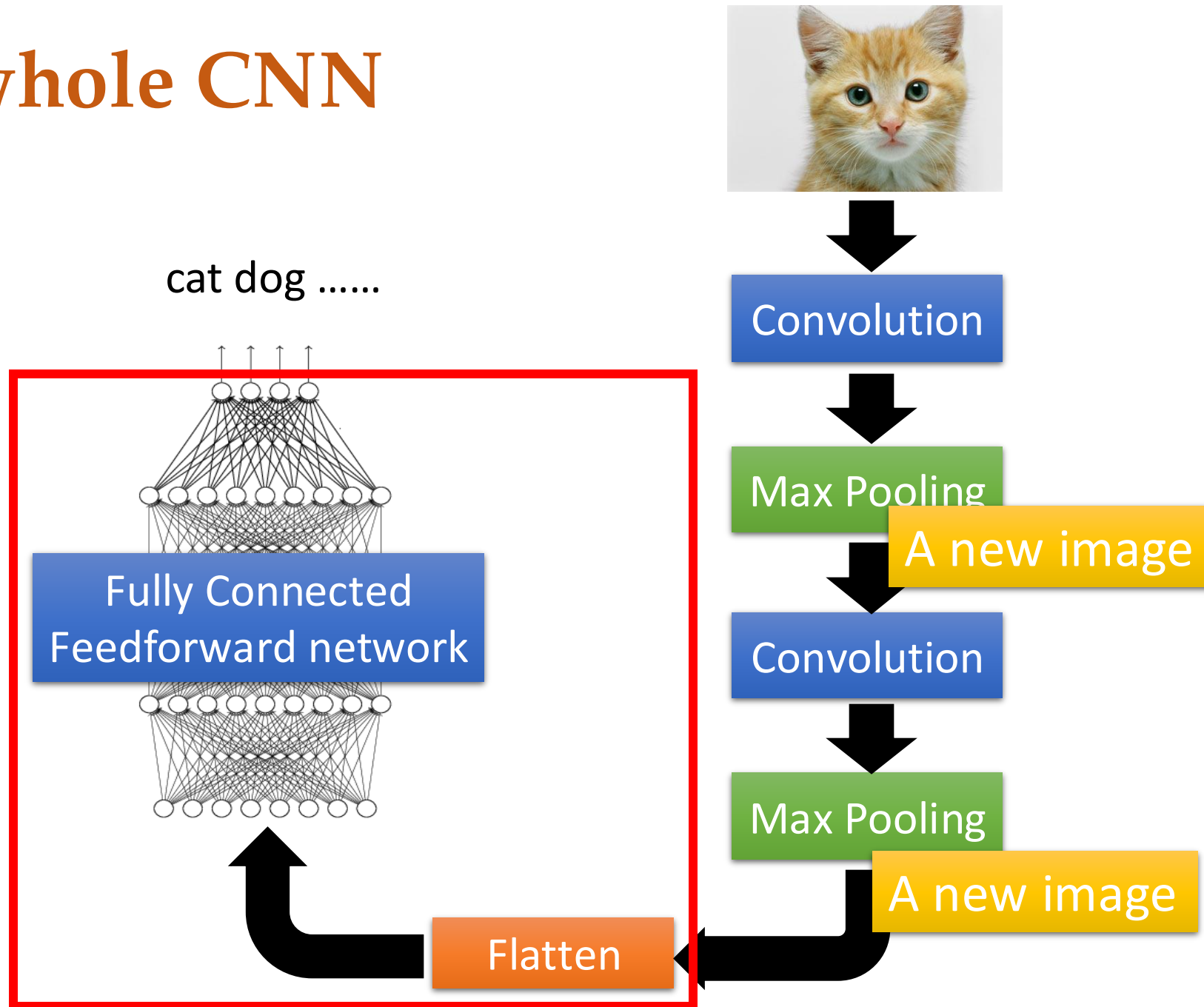
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

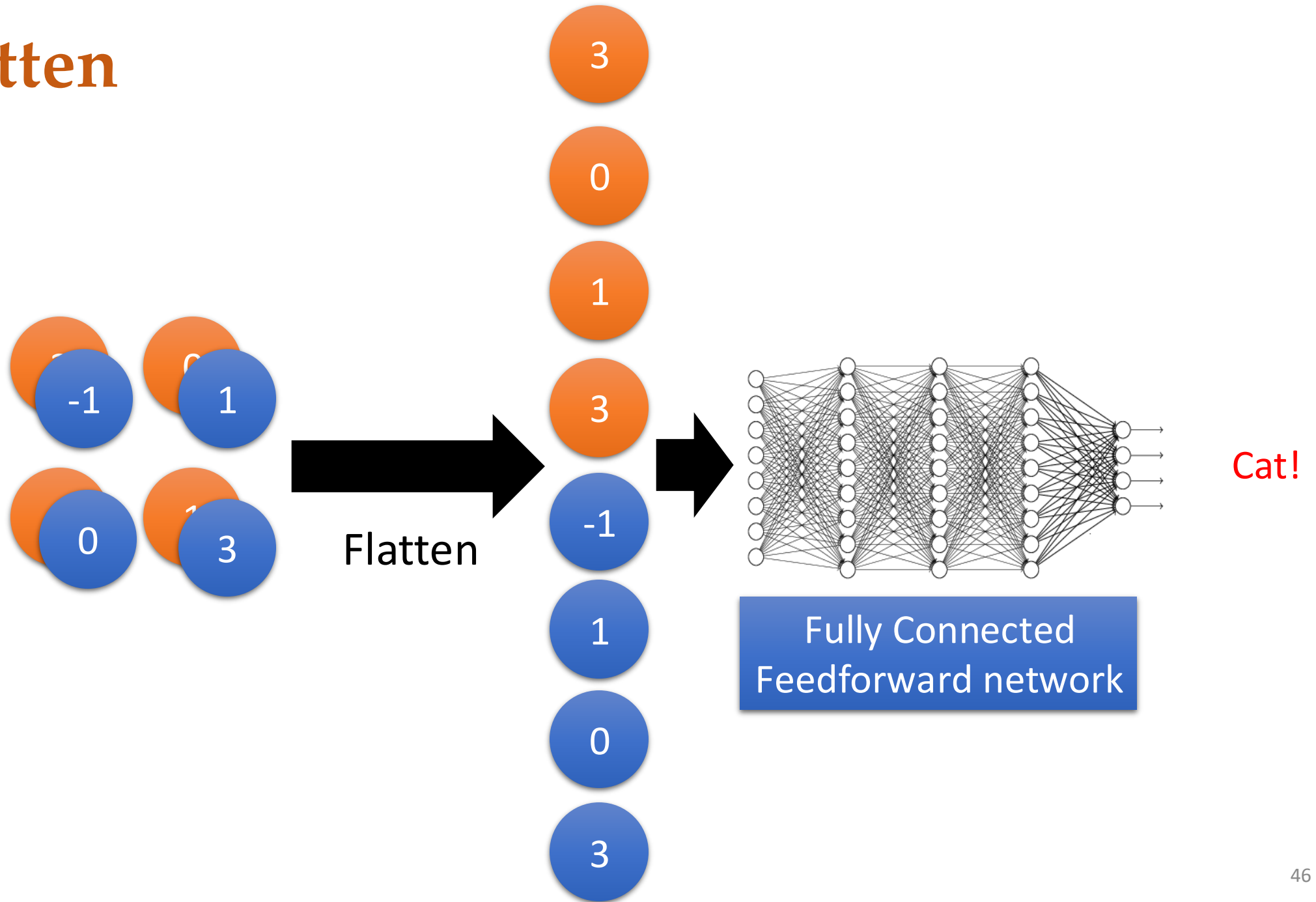
CNN – max pooling



The whole CNN



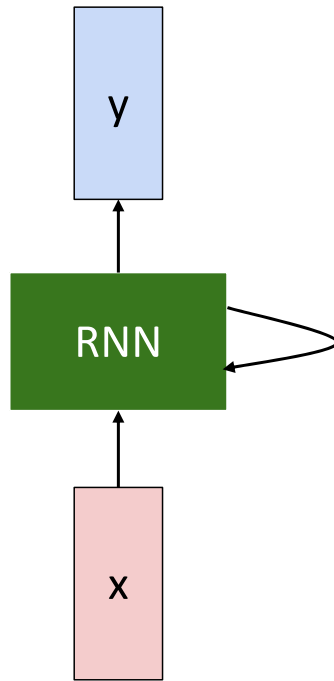
Flatten



Agenda

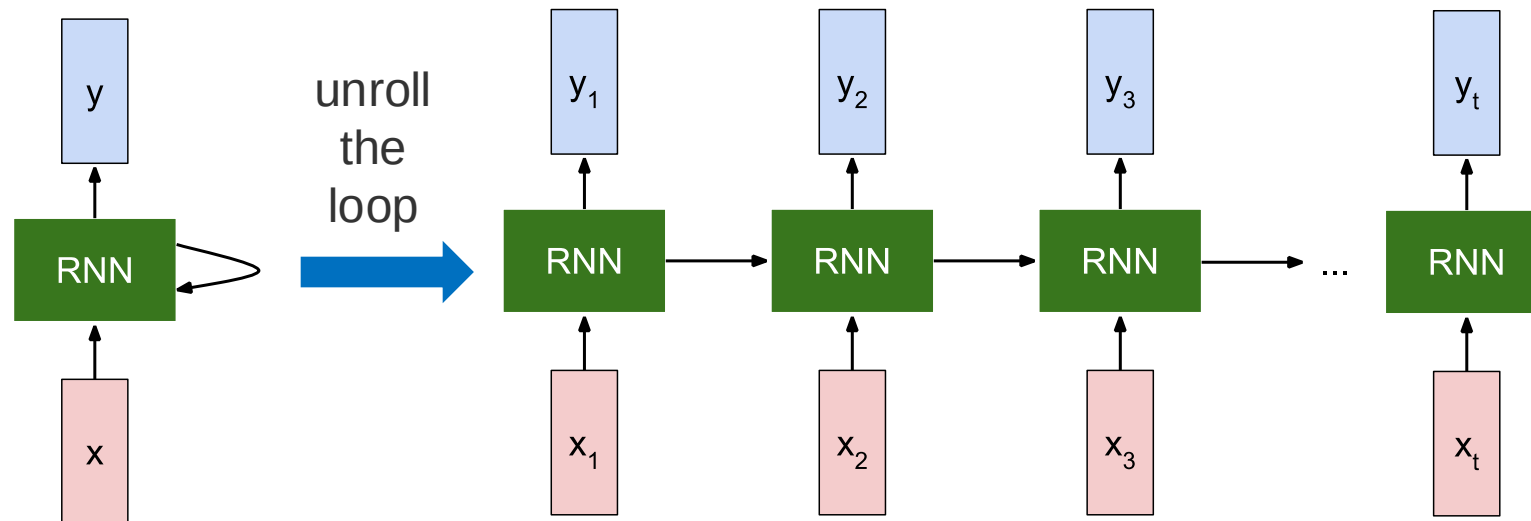
- Linear Regression
- Neural Networks
- CNN
- RNN

How does RNN work



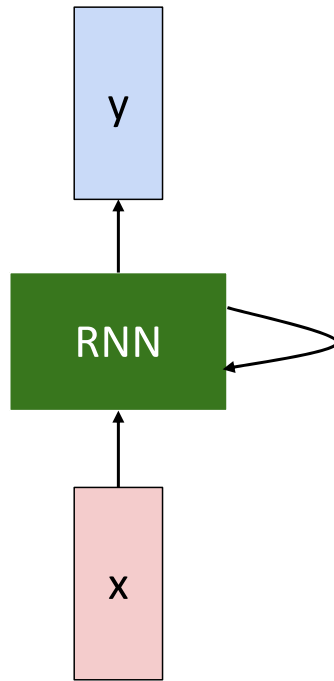
Key idea: RNNs have an “**internal state**” (hidden state) that is updated as a sequence is processed

Computation of RNN



A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor

Computation of RNN



We can process a sequence of vectors x by applying a recurrence formula at every time step:

new
state

activation
function

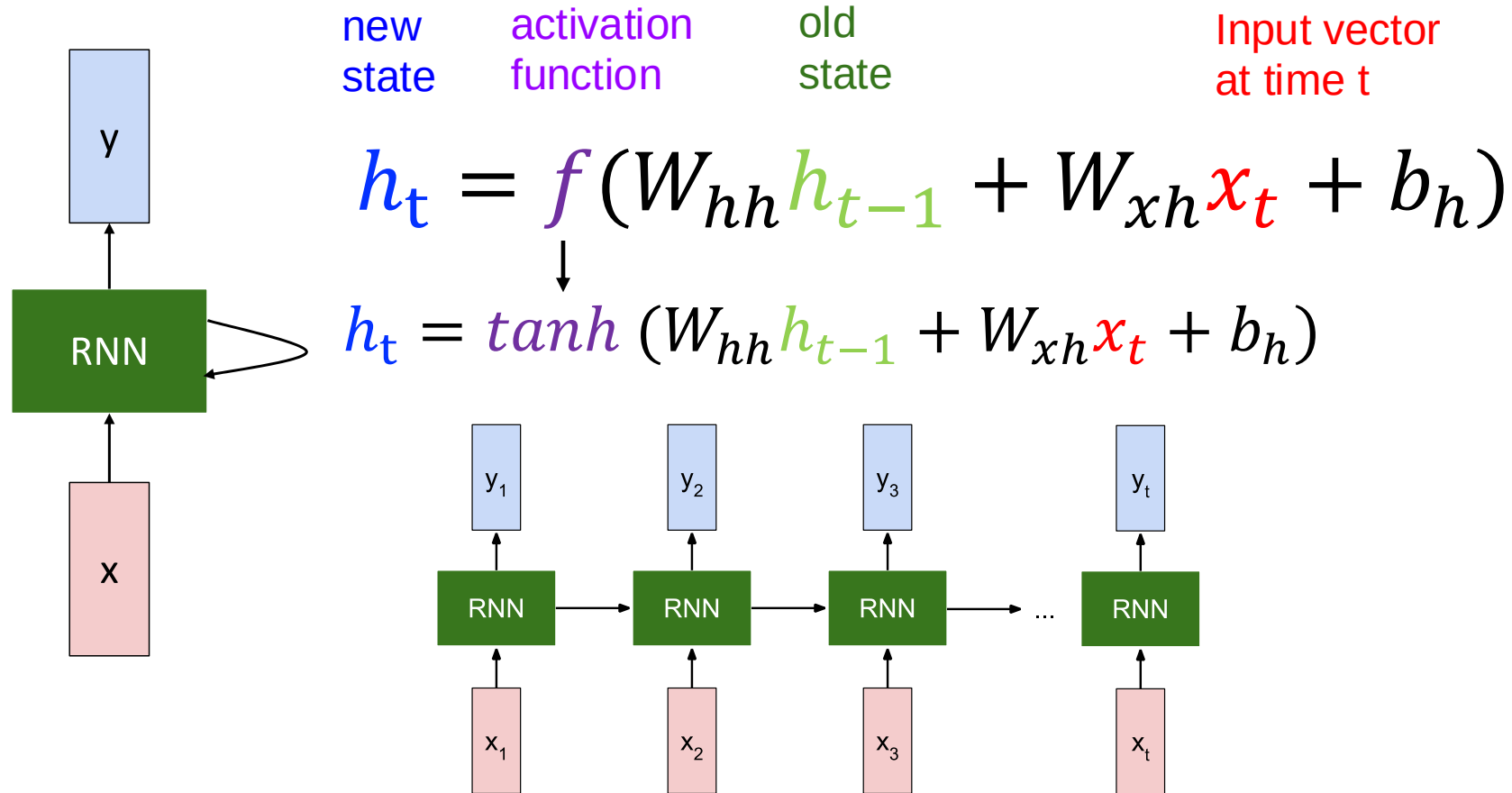
old
state

Input vector at
time t

$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

Notice: the same function and the same set of parameters are used at every time step.

Computation of RNN



Recurrent Neural Networks

